Ternava

A **1.58-bit sentence-embedding model** *packaged as a WASM module. Runs anywhere JavaScript runs. Fits in 5 MB. Embeds a sentence in under 2 milliseconds on an M-series Mac. Inspired by Microsoft's* **BitNet b1.58**.

What Ternava is

A small sentence encoder — given a piece of text, returns a 384-dimensional unit vector representing its semantic content. Two of these vectors can be compared via cosine similarity to get a meaningful "how related are these?" score, which is the foundation for semantic search, FAQ matching, clustering, deduplication, etc.

How is Ternava *different?*

Runs anywhere JavaScript runs

- No network call, no GPU, no ML framework runtime
- In a browser tab, a Cloudflare Worker, a Node.js script, an Edge function — anywhere with a WebAssembly runtime

Ships as one file

- ~5 MB `.wasm`, ~3 MB gzipped on the wire
- No external dependencies, no model fetch at runtime, no warm-up server

Built on three technical choices, stacked

- Training the model to be aggressively quantizable — distillation + QAT
- Packing the model into a custom binary format
- Inference engine in Rust → WASM, hand-tuned for parallel CPU instructions

Why on-device

The default mode for "I need an embedding model in my app" is to call a remote endpoint (e.g. OpenAI's embedding API). That works, but it ties every UX to a network call:

- **Latency**: 100–300 ms per round-trip — search-as-you-type feels laggy.
- **Cost**: every call is a billable
- **Privacy**: queries and documents leave the user's device.
- **Availability**: no offline; flaky networks break the UX.

Running embedding inference *on the user's device* eliminates all four. The catch: encoders are usually too big (25–100 MB), too slow on CPU (hundreds of ms), and need an ML runtime. Ternava pushes those numbers low enough that "ship the model with the page" becomes a sane default.

The goal: decent-quality embeddings, produced cheaply, on the user's device.

Results

These numbers describe the current shipped state (`emb_ternary` build, M4 Max, Node v20 + V8).

Dimension	*Number*	*Notes*
Single `embed()` latency (p50)	`1.82 ms`	Time to embed one sentence
Bundle size (`emb_ternary`)	`5.4 / 3.2 MB`	Full `.wasm` (model + tokenizer + engine), raw / gzipped
Bundle size (`emb_int8`)	`11.0 / 8.7 MB`	Same, int8 embedding-table variant, raw / gzipped
Throughput (single-threaded)	`~550 /s`	Single-threaded embed rate
Quality (teacher fidelity)	`0.83`	Spearman — student vs MiniLM-L6 teacher pairwise rank correlation
Quality (downstream retrieval)	`0.45`	NDCG@10 — SciFact retrieval, 300 queries × 5,183 docs

Table 1. Current shipped results for the `emb_ternary` build.

At 1.82 ms per embedding, a query-time fan-out over **a corpus of ~500 documents** completes in under a second — feels like "instant search."

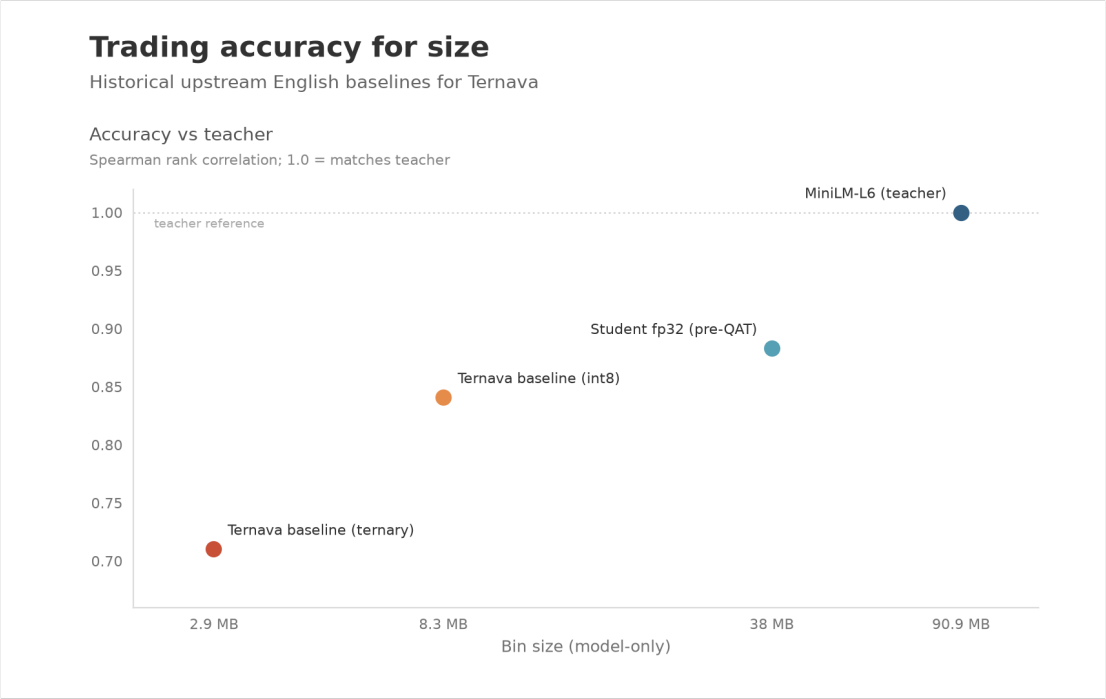


Figure 1. The size / accuracy trade-off. Teacher fidelity (Spearman vs MiniLM-L6) plotted against model size for the teacher, the fp32 student, and both Ternava ship variants. Ternava (`emb_ternary`) reaches ~30× compression for a modest fidelity drop.

> **Charts deferred.** *Bundle size vs comparable on-device encoders (MiniLM ONNX int8, Universal Sentence Encoder Lite, etc.).*

The three levers

The conceptual foundation is **BitNet b1.58** (Microsoft, 2024) — transformer weights restricted to just three values, `{-1, 0, +1}`, can match the quality of full-precision (fp32) baselines. This finding reshapes the cost model of inference where the core operation in any transformer forward pass is the matrix multiply (matmul). If every weight is ternary, the matmul simplifies to integer adds and subtracts (with no multiplication at all). Everything Ternava does is downstream of that insight.

① Training	**Distillation + QAT.** A small student is taught by a stronger teacher *while being trained as a ternary model from the start* — the compression is baked into training.
② Packing	**Ternary bit format.** Each weight goes from 32 bits → 2 bits. The whole model + tokenizer + engine code ships as one ~5 MB `.wasm` (3 MB gzipped), with no external dependencies and no model fetch at runtime.
③ Inference	**Rust → WASM, SIMD-accelerated.** A from-scratch forward pass written in Rust and compiled to WebAssembly, hand-tuned to use modern CPUs' parallel-arithmetic instructions (`simd128`). Runs at near-native CPU speed inside any WASM runtime — no GPU, no ML framework.

1. Training: making a small model good

Architecture

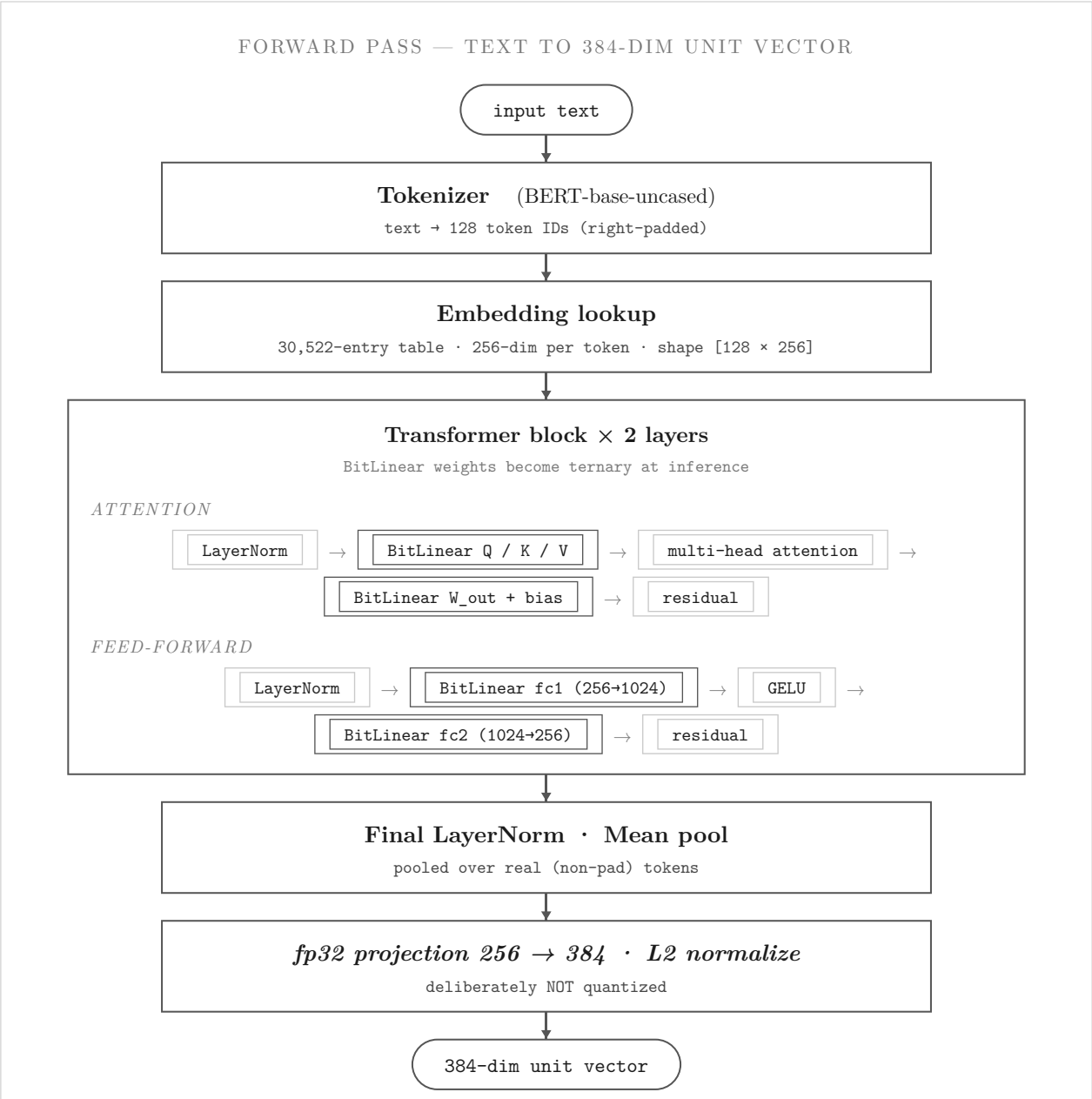


Figure 2. Model architecture and forward pass. Shaded-bold blocks are the BitLinear matrices whose weights are ternarized at inference time.

Concretely:

- **Architecture**: 2-layer transformer, `d_model=256`, 4 attention heads, FFN dim 1024, output 384
- **~9M parameters total** — the 12 BitLinear matrices compress to **~0.4 MB** after ternary packing
- **The embedding table dominates the size**, not the transformer

Distillation + QAT (the joint trick)

Ternava merges two ideas:

Distillation. A frozen teacher model (`all-MiniLM-L6-v2`) provides the target signal. For each training sentence, the teacher produces a 384-dim target vector. The student is trained to match these vectors via cosine loss. This is the source of the student's semantic knowledge — the teacher embeds the knowledge of its much larger training corpus into compact targets.

Quantization-Aware Training (QAT). Every training forward pass *simulates* the ternary quantization the deployed model will use, so the model adapts to the constraint as it learns. The loss the model optimizes against already reflects the cost of being ternary.

The two together. The student is trained *as the quantized model it will become*, against a teacher that knows what good embeddings look like. The model learns to be good *while being ternary.* This is the only reason you can shrink a sentence encoder to 5 MB without semantic collapse.

Loss function

```
loss = 1.0 × cosine_distill_loss(student, teacher)
     + 0.15 × contrastive_repulsion(student_batch)
```

- The **distillation term** is the main signal: 1 − cos(student_emb, teacher_emb), averaged across the batch. Drives the student to match the teacher.
- The **contrastive term** is a guardrail against embedding collapse — under quantization the student can degenerate to "map everything near the teacher's average point." The contrastive term penalizes high pairwise similarity between *different* sentences in the same batch, pushing the embeddings to fill the unit sphere rather than cluster.

Training corpus & recipe

The micro-tier rigorous run uses **1 million samples** drawn from a three-source mix:

- 60% MS MARCO (v2.1) — Bing search queries, short and question-like
- 25% sentence-transformers/gooaq — Google Q&A questions
- 15% sentence-transformers/quora-duplicates — paraphrase pairs

40 epochs total. The first 5 are an **fp32 warmup**; at epoch 6 ternary quantization activates and the model adapts to it over the remaining 35 epochs.

> **Chart deferred.** *Training and validation loss curves with the `lambda=0 → 1` boundary marked at epoch 5 — visualizes the small loss spike and recovery.*

Training results

The trained QAT checkpoint is evaluated on three axes:

- **Quantization gap** — quality lost going from fp32 to ternary at the same architecture.
- **Embedding diversity** — whether the contrastive guardrail successfully prevents collapse under quantization.
- **Inference parity** — whether the packed `.bin` produces bit-faithful outputs against the Python reference.

Concrete numbers: TBD.

Quality assessment results: TBD — see §4 Quality for the eval framework.

2. Packing: 36 MB → 5 MB

The resulting PyTorch checkpoint is `~36 MB` (mostly the float32 embedding table). The shipped `.bin` is `~5 MB`. Compression happens in the packer.

Why ternary weights work

During training, each weight is stored as **fp32 (32 bits)** but the forward pass only ever uses its ternary projection. Packing throws those "shadow" fp32 values away and keeps only the 2-bit ternary code each weight had converged to. **Those ternary codes are what the forward pass actually uses for inference** — the fp32 shadow was just scaffolding for gradient descent during training, never read at inference time.

Each weight maps to a 2-bit code:

Code	*Ternary value*
`00`	0
`01`	+1
`10`	−1
`11`	*reserved*

Table 2. 2-bit ternary code assignment.

Four codes pack into one byte:

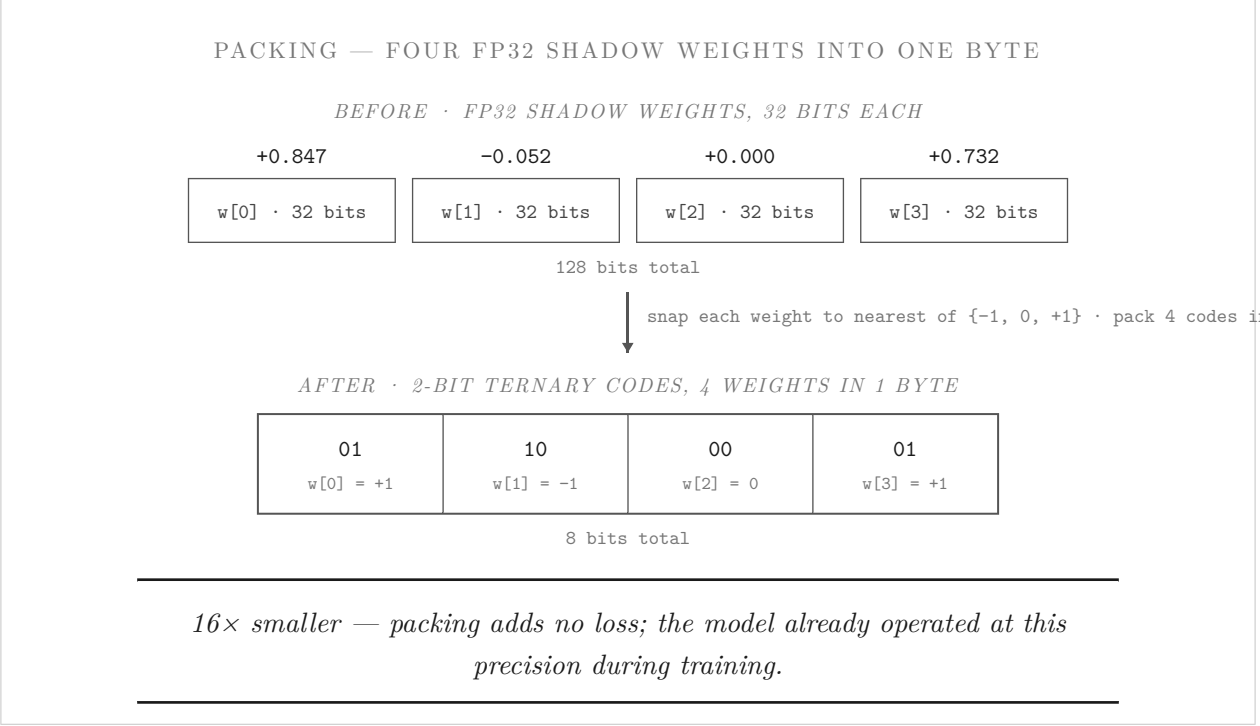


Figure 3. Weight packing. Four 32-bit shadow weights collapse to four 2-bit codes in a single byte.

Each matrix also ships with one `fp32` *scale*. Ternary encodes the *sign* of each weight; the scale tells us how *big* the weights actually are. The two are recombined at inference.

What ships in the `.bin`

Section	*emb_int8*	*emb_ternary*
32-byte header (magic, version, dims)	32 B	32 B
Token embedding table (30,522 × 256)	7.45 MB	1.86 MB
Embedding per-row scales	122 KB	122 KB
BitLinear weights × 12 (all matrices)	384 KB	384 KB
BitLinear scales + biases	~7 KB	~7 KB
LayerNorm parameters	~10 KB	~10 KB
fp32 output projection	384 KB	384 KB
SHA256 (trailing 32 bytes)	32 B	32 B
Total `.bin`	~8.4 MB	~2.75 MB

Table 3. Section breakdown of the two build formats. The embedding table is the only section that differs.

The packer also embeds the tokenizer (~0.68 MB), and the WASM code itself is ~2 MB. All of this gets `include_bytes!`-ed into the final `.wasm`, producing the 11 MB / 5.4 MB ship artifacts.

.bin byte layout

How the sections are arranged in the file, in order from the first byte to the last:

Offset	*Section*	*Contents*
0	Header (32 B)	magic, version, formats, dims — see field map below
32	Token embedding table	followed by per-row scales
·	Transformer layer 0	attention: LN | Q | K | V | W_out · FFN: LN | fc1 | fc2
·	Transformer layer 1	same shape as layer 0
·	Final LayerNorm	—
·	fp32 output projection	256 → 384, not quantized
EOF−32	SHA256 trailer (32 B)	integrity check over preceding bytes

Table 4. Top-level `.bin` section layout.

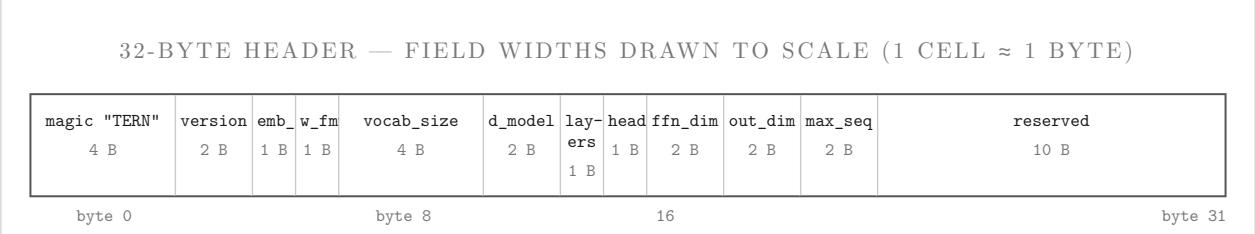


Figure 4. The 32-byte header. Each field's box width is proportional to its size in bytes. Values: `version=1`, `w_fmt=ternary`, `vocab_size=30522`, `d_model=256`, `layers=2`, `heads=4`, `ffn_dim=1024`, `out_dim=384`, `max_seq=128`.

The engine walks this layout once at init: parses the header, computes section offsets, and pre-decodes the small fp32 sections (LayerNorm params, scales, biases, projection).

3. Inference engine: from .bin to vector

Why Rust → WebAssembly (WASM). Rust delivers near-native CPU performance — first-class **SIMD** (*Single Instruction, Multiple Data*) primitives, and deterministic memory management with no garbage-collection. WASM gives a portable, sandboxed execution target that runs unmodified in every JavaScript runtime — browser tab, Node script, Cloudflare Worker, Edge function — with no install step. Together they're the only combination that delivers near-native CPU speed *and* universal portability.

What we built. A from-scratch forward pass that re-implements the model's training-time math in Rust, parity-tested against the Python reference. On top of that we added inference-only optimizations the training code doesn't need — taking a single embed query to 1.82 ms.

The engine is a Rust crate that compiles to `wasm32-unknown-unknown` via wasm-pack. Public surface is three functions: `embed(text) → Vec<f32>`, `tokenize(text) → Vec<u32>`, and `config_summary() → String`.

What one `embed()` call does

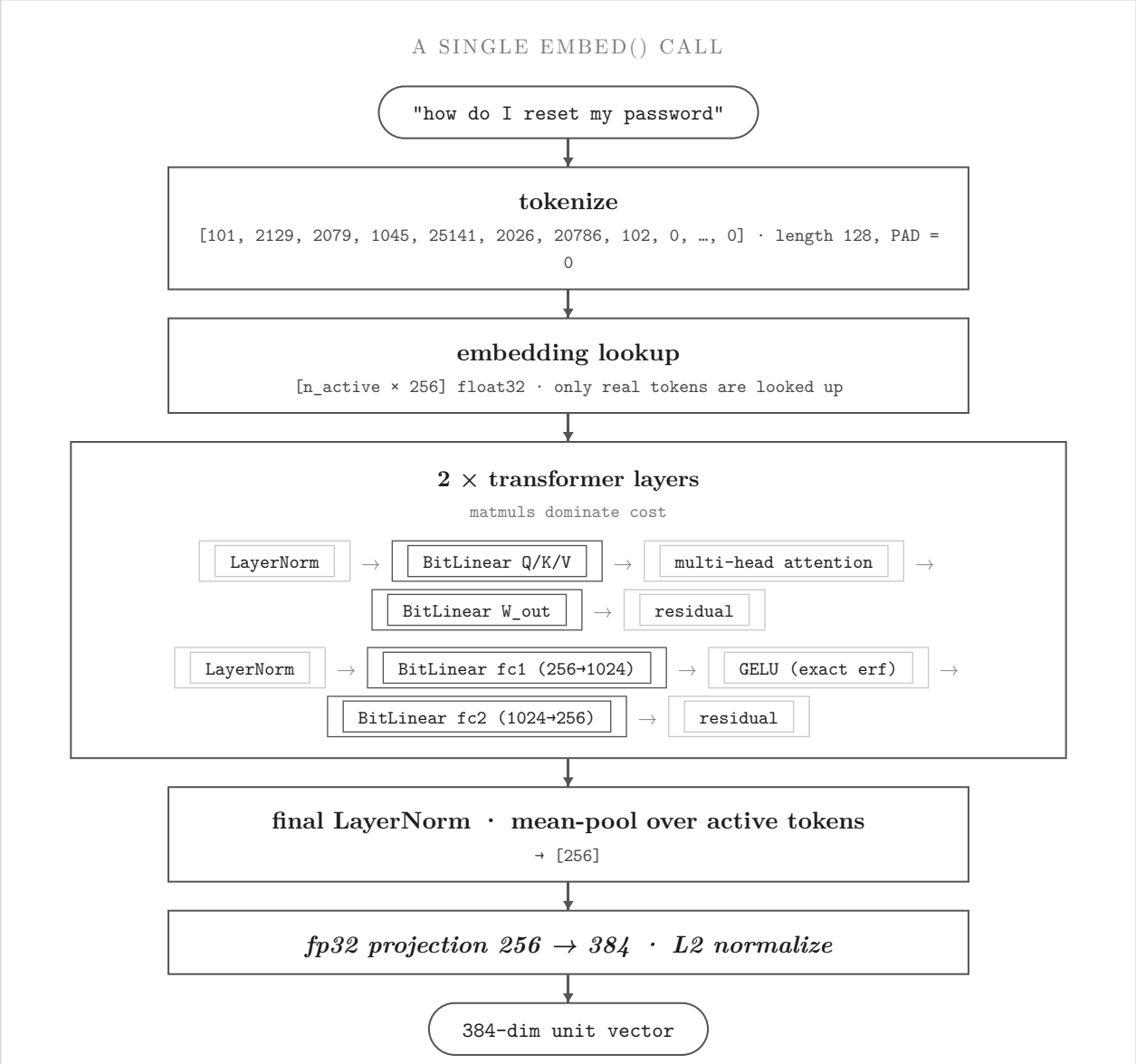


Figure 5. Execution trace of one `embed()` call, from raw text to the normalized 384-dim output vector.

What makes the engine fast

Three inference-only optimizations sit on top of the forward pass; together they're what land the engine at **1.82 ms p50**:

- **Padding skip.** Tokenizers right-pad every input to a fixed length (128 tokens), but real inputs rarely use anywhere near that. The engine iterates over the actual tokens only, skipping the padded positions the attention mask would zero out anyway. The shorter the input, the bigger the saving.
- **Pre-unpacked weights.** At engine init, the 2-bit packed ternary weights are expanded into `i8` (`-1`, `0`, `+1` stored as single bytes). This trades a little memory overhead to skip the conditional unpack step inside the matmul — the inner loop becomes a straight `i8 × i8` multiply-accumulate the CPU can run at full speed.

- **Explicit SIMD intrinsics.** The matmul inner loop is hand-written against WASM `simd128` (16-wide parallel multiply-accumulate). Baking SIMD into the bytecode guarantees the speedup runs on every compliant runtime — V8, Wasmtime, JavaScriptCore.

Portability and where it fits

The usual pain with shipping ML inference is platform-specific glue. A PyTorch runtime install runs into hundreds of MB. ONNX Runtime needs a build per target CPU (and a different one per GPU). CUDA wants drivers; Metal wants Xcode; mobile pulls in CoreML or TFLite. Every deployment target spawns its own bloated artifact, its own dependency hell, its own install ritual — and that pain multiplies across every OS, CPU, and chip vendor you need to support.

Ternava sidesteps that. One `.wasm` file runs in every compliant runtime — Node ≥16.4, modern browsers, Cloudflare Workers, Vercel Edge, Deno, Bun — with the SIMD opcodes baked into the bytecode, so speed travels with the file. No native binaries to build per platform, no GPU drivers, no install step.

Sweet spot: applications that already host a JavaScript runtime — browser tabs, Edge functions, serverless Workers, Node services. Anywhere shipping a native ML runtime would be the real barrier.

---

4. Quality: what we actually measure

Ternava is evaluated at three levels of rigor, in increasing order:

- **Smoke test** (every build) — A small set of hand-picked sentence pairs (identity, paraphrase, negative anchor, gibberish) run through the engine to catch "model loaded but produces garbage" failure modes that aggregate metrics would mask. Fast, qualitative, runs after every build.
- **Per-checkpoint formal eval** — A full evaluation harness against held-out benchmarks: pairwise rank correlation vs the teacher (`test/spearman`), STS-Benchmark (`stsb/{spearman,pearson}`), retrieval (`retrieval/ndcg@10` on SciFact). Run per checkpoint to pick the right one to ship.
- **Quantization gap** — The above metrics computed against a same-architecture **fp32 baseline** trained on the same data, isolating how much quality was lost specifically to ternarization vs. inherent to the small architecture. The honest answer to "what did the compression cost us?"

The quantization gap, visualized

QAT (shipped, ternary) vs. a same-architecture **fp32 baseline** trained on the same data, on the held-out test set:

Metric	*fp32 baseline*	*QAT (shipped)*	*Δ vs fp32*
`test/spearman` (teacher fidelity)	0.86	0.83	−0.035
`NDCG@10` (SciFact retrieval)	0.443	0.448	+0.005 ★
`recall@5` (SciFact retrieval)	0.492	0.518	+0.026 ★

Table 5. Quantization gap. ★ marks metrics where the ternary model matches or exceeds the fp32 baseline.

The teacher-fidelity gap is small (~3.5 Spearman points). On the actual downstream retrieval task — the metric that matters in production — the QAT model edges out its fp32 ruler. Within noise, but a strong signal that the ternary constraint isn't costing us anything on real use cases. STS-Benchmark and additional benchmarks still pending.

5. Packaging and distribution

Work in progress. Today the engine builds to `engine/pkg/` and is consumed locally for testing; the proper npm package is the next phase.

Packaging plan (multi-target builds, exports routing, async-init wrapper, known runtime gotchas): see docs/tern-bundling.md.